Course Outline: Working with Arrays and Collections

Title: Working with Arrays and Collections in TypeScript **Prerequisite:** Functions in TypeScript (Week 3, Day 8) **Target Audience:** Beginner developers learning TypeScript fundamentals **Total Lessons:** 20 **Estimated Total Length:** ~11,825 words **Format:** Practical (58% hands-on)

Course Description

Arrays are one of the most fundamental data structures in programming. This course teaches students how to work with collections of data in TypeScript, from basic array operations to advanced searching and sorting algorithms. Students will learn to store, access, manipulate, and search through data efficiently.

Building on the functions knowledge from the previous lesson, students will combine iteration patterns with array methods to solve real-world data problems. The course progresses from simple one-dimensional arrays to multi-dimensional matrices, preparing students for object-oriented programming in the next lesson.

By the end of this course, students will confidently work with arrays, understand when to use different iteration approaches, and apply efficient searching and sorting algorithms to solve practical problems.

Course Philosophy

This course emphasizes:

- **Practical iteration patterns**: Learn by doing, not memorizing syntax
- **Understanding before optimization**: Master linear search before binary search
- **Visual thinking**: See arrays as containers of related data
- **Incremental complexity**: Start with simple arrays, build to matrices
- **Path of least resistance**: Use built-in methods when available, understand algorithms when needed

---

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - Array Fundamentals	4	~2,300
02 - Iterating Through Arrays	3	~1,800
03 - Multi-Dimensional Arrays	3	~1,800
04 - Sorting and Searching	5	~3,000
05 - Practice and Assessment	3	~2,050
06 - Conclusion	1	~425
Total	**20**	**~11,825**

Section 00: Welcome

00.0 Introduction to Collections 🕮 (~450 words)

Learning Objectives:

- Understand what arrays are and why they're essential in programming
- Recognize situations where arrays solve real-world problems
- Connect arrays to previous programming concepts (variables, loops, functions)
- Preview the journey from simple arrays to complex data manipulation

Content Outline:

- The problem: managing multiple related values
- What arrays are: ordered collections of elements
- Why arrays matter: storing lists, processing data sets, building applications
- Course roadmap: from basics to searching algorithms
- Connection to previous lessons: functions + arrays = powerful tools

Transitions: Next, we'll dive into array fundamentals, learning how to create and access array elements.

Key Principles:

- Arrays store multiple values under a single variable name
- Each element has a position (index) starting at 0
- Arrays are the foundation for managing collections of data
- Understanding arrays unlocks data-driven programming

Examples:

- Student grades: Instead of `grade1`, `grade2`, `grade3`... use `grades[0]`, `grades[1]`, `grades[2]`
- Shopping cart items: Store product names in a single array
- Temperature readings: Track daily temperatures in an ordered list
- User list: Manage multiple usernames efficiently

Section 01: Array Fundamentals

01.0 What Are Arrays? 🕮 (~500 words)

Learning Objectives:

- Define arrays and their purpose in programming
- Understand array syntax and structure in TypeScript
- Identify the difference between individual variables and array collections
- Recognize when to use arrays vs. individual variables

Content Outline:

- Definition: arrays as ordered, indexed collections
- Syntax: square brackets, comma-separated values
- Zero-based indexing: why arrays start at 0

- Array length: understanding the `.length` property
- Type consistency: homogeneous arrays (same type elements)

Transitions: From previous lesson: Functions helped us organize code blocks. Now arrays help us organize data blocks. Next, we'll practice creating and accessing arrays hands-on.

Key Principles:

- Arrays group related data under one name
- Index positions start at 0, not 1
- Length property always equals the number of elements
- TypeScript arrays should contain elements of the same type

Examples:

```
// Individual variables (inefficient)
let student1 = "Ana";
let student2 = "Carlos";
let student3 = "Maria";

// Array (efficient)
let students: string[] = ["Ana", "Carlos", "Maria"];

// Accessing elements
console.log(students[0]); // "Ana"
console.log(students.length); // 3
```

01.1 Creating and Accessing Arrays 💻 (~600 words)

Learning Objectives:

- Create arrays using literal notation
- Access array elements using bracket notation
- Use the `.length` property effectively
- Modify existing array elements

Content Outline:

- Array creation: literal syntax `[]`
- Type annotations: `string[]`, `number[]`, `boolean[]`
- Accessing elements: bracket notation `array[index]`
- Reading vs. writing: getting and setting values
- Common mistakes: accessing non-existent indices

Transitions: You've learned to create and read arrays. Now we'll practice modifying them by adding and removing elements.

Key Principles:

- Use literal notation `[]` for creating arrays
- Always specify type in TypeScript: `number[]`
- Accessing out-of-bounds returns `undefined`
- Assignment changes element values: `array[0] = newValue`

Examples:

```
// Different array types
let numbers: number[] = [10, 20, 30, 40];
let names: string[] = ["Alice", "Bob", "Charlie"];
let flags: boolean[] = [true, false, true];

// Accessing and modifying
console.log(numbers[1]); // 20
numbers[1] = 25;
console.log(numbers[1]); // 25
```

Hands-On Exercise: Task: Create a personal data collection

1. Create an array called `favoriteFoods` with 5 string elements
2. Create an array called `scores` with 4 number elements
3. Print the first and last element of each array
4. Change the third element of `scores` to a new value
5. Print the updated `scores` array

Success Criteria:

- Both arrays created with correct types
- Elements accessed using correct indices
- Modified element shows new value
- All console outputs display correctly

01.2 Adding and Removing Elements 💻 (~650 words)

Learning Objectives:

- Add elements to arrays using `.push()` and `.unshift()`
- Remove elements using `.pop()` and `.shift()`
- Understand how these methods affect array length
- Choose the appropriate method for different scenarios

Content Outline:

- Adding to end: `.push()` method
- Adding to beginning: `.unshift()` method
- Removing from end: `.pop()` method
- Removing from beginning: `.shift()` method
- Return values: what these methods give back

- Length changes: tracking array size

Transitions: Now that you can add and remove elements, the next lesson covers array properties and useful methods for everyday operations.

Key Principles:

- `.push()` adds to end, `.pop()` removes from end
- `.unshift()` adds to beginning, `.shift()` removes from beginning
- These methods modify the original array (mutation)
- `.pop()` and `.shift()` return the removed element

Examples:

```
let tasks: string[] = ["Email", "Meeting"];

// Adding elements
tasks.push("Report"); // ["Email", "Meeting", "Report"]
tasks.unshift("Coffee"); // ["Coffee", "Email", "Meeting", "Report"]

// Removing elements
let last = tasks.pop(); // "Report", array: ["Coffee", "Email", "Meeting"]
let first = tasks.shift(); // "Coffee", array: ["Email", "Meeting"]
```

Hands-On Exercise: Task: Build a queue management system

1. Create an empty array called `customerQueue`
2. Add 3 customer names using `.push()`
3. Print the queue and its length
4. Remove the first customer using `.shift()` and print who was served
5. Add 2 more customers to the end
6. Remove the last customer using `.pop()` and print who left
7. Print the final queue

Success Criteria:

- Queue starts empty and grows correctly
- `.shift()` removes from front (FIFO - first in, first out)
- `.pop()` removes from back
- Final queue contains expected customers
- Console shows clear progression

01.3 Essential Array Properties and Methods 🕮 (~550 words)

Learning Objectives:

- Use `.length` property for dynamic operations
- Apply `.indexOf()` to find element positions
- Check element existence with `.includes()`

- Understand when to use each method

Content Outline:

- `.length`: counting elements, loop limits
- `.indexOf()`: finding first occurrence, returns index or -1
- `.includes()`: boolean check for element existence
- `.slice()`: extracting portions without mutation
- `.concat()`: combining arrays without mutation

Transitions: With these fundamental methods mastered, we're ready to explore iteration—the key to processing every element in an array.

Key Principles:

- `.length` is always one more than the highest index
- `.indexOf()` returns -1 if element not found
- `.includes()` returns boolean (true/false)
- Some methods mutate (push, pop), others don't (slice, concat)

Examples:

```
let fruits: string[] = ["apple", "banana", "orange", "banana"];

// Finding elements
console.log(fruits.indexOf("banana")); // 1 (first occurrence)
console.log(fruits.includes("grape")); // false

// Non-mutating operations
let citrus = fruits.slice(2, 4); // ["orange", "banana"]
let combined = fruits.concat(["mango"]); // doesn't change fruits

// Using length
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

Section 02: Iterating Through Arrays

02.0 Looping Through Arrays 🕮 (~500 words)

Learning Objectives:

- Understand why iteration is essential for array processing
- Use `for` loops to traverse arrays
- Use `while` loops for conditional array iteration
- Recognize when to use each loop type

Content Outline:

- Why iterate: processing every element
- `for` loop pattern: index-based traversal
- Loop condition: `i < array.length`
- `while` loops: condition-based iteration
- Common patterns: counting, summing, finding

Transitions: From functions lesson: loops were introduced. Now we apply them specifically to arrays. Next, we'll explore modern array iteration methods that simplify common tasks.

Key Principles:

- Iteration means visiting each element sequentially
- `for` loops work best when you need the index
- Always use `i < array.length`, never hardcode limits
- Loop variables (`i`) should be declared with `let`

Examples:

```
let prices: number[] = [10.99, 5.5, 12.0, 8.25];

// For loop - classic pattern
for (let i = 0; i < prices.length; i++) {
  console.log(`Item ${i + 1}: $${prices[i]}`);
}

// While loop - conditional processing
let total = 0;
let i = 0;
while (i < prices.length && total < 20) {
  total += prices[i];
  i++;
}
```

02.1 Array Iteration Methods 💻 (~650 words)

Learning Objectives:

- Use `.forEach()` for simple iteration
- Apply `.map()` to transform arrays
- Filter arrays with `.filter()`
- Choose the appropriate iteration method for each task

Content Outline:

- `.forEach()`: executing code for each element
- `.map()`: transforming arrays into new arrays
- `.filter()`: selecting elements that meet criteria
- `.reduce()`: brief introduction for accumulation
- Method chaining: combining operations

Transitions: These methods simplify iteration. Next, we'll practice real-world iteration patterns combining these techniques.

Key Principles:

- `.forEach()` for side effects (printing, counting)
- `.map()` creates a new array (doesn't mutate original)
- `.filter()` returns elements matching a condition
- Modern methods are more readable than traditional loops

Examples:

```
let numbers: number[] = [1, 2, 3, 4, 5];

// forEach - execute for each
numbers.forEach((num) => console.log(num * 2));

// map - transform each element
let doubled: number[] = numbers.map((num) => num * 2);
// [2, 4, 6, 8, 10]

// filter - select matching elements
let evens: number[] = numbers.filter((num) => num % 2 === 0);
// [2, 4]
```

Hands-On Exercise: Task: Process a student grades list

1. Create an array `grades` with 8 number values (0-100 range)
2. Use `.forEach()` to print each grade with "Grade: X"
3. Use `.map()` to create a new array with each grade increased by 5 points
4. Use `.filter()` to create an array of only passing grades (≥60)
5. Print the original, boosted, and passing arrays

Success Criteria:

- Original array unchanged after operations
- Boosted array has all values +5
- Passing array contains only grades ≥60
- Each method used correctly

02.2 Practical Iteration Patterns 💻 (~650 words)

Learning Objectives:

- Find maximum and minimum values in arrays
- Calculate sums and averages
- Count elements matching specific criteria
- Build new arrays from existing data

Content Outline:

- Finding max/min without built-in methods
- Summing array elements
- Calculating averages
- Counting occurrences
- Building filtered/transformed results

Transitions: With iteration mastered, we're ready to tackle multi-dimensional arrays—arrays containing other arrays.

Key Principles:

- Initialize accumulators before loops (max, sum, count)
- Use meaningful variable names in iterations
- Test edge cases (empty arrays, single elements)
- Combine methods for complex operations

Examples:

```
let temperatures: number[] = [72, 68, 75, 71, 69, 73];

// Find maximum
let max = temperatures[0];
for (let i = 1; i < temperatures.length; i++) {
  if (temperatures[i] > max) {
    max = temperatures[i];
  }
}

// Calculate average
let sum = 0;
temperatures.forEach((temp) => (sum += temp));
let average = sum / temperatures.length;

// Count above threshold
let hotDays = temperatures.filter((temp) => temp > 70).length;
```

Hands-On Exercise: Task: Shopping cart analysis

1. Create an array `cart` with 6 product prices (numbers)
2. Find the most expensive item (max)
3. Find the cheapest item (min)
4. Calculate the total cost (sum)
5. Count how many items cost more than $10
6. Create an array of discounted prices (10% off each item)
7. Print all results with labels

Success Criteria:

- Max and min correctly identified
- Total calculated accurately
- Count matches condition
- Discount array has all prices reduced by 10%
- Output clearly labeled

Section 03: Multi-Dimensional Arrays

03.0 Introduction to Matrices 🕮 (~550 words)

Learning Objectives:

- Understand what matrices (2D arrays) are
- Visualize matrices as rows and columns
- Access matrix elements using double indices
- Recognize real-world applications of matrices

Content Outline:

- Definition: arrays of arrays
- Visualization: rows and columns
- Syntax: nested square brackets
- Accessing elements: `matrix[row][column]`
- Common uses: grids, tables, game boards

Transitions: You've mastered 1D arrays. Matrices extend this concept to two dimensions. Next, we'll practice creating and manipulating matrices.

Key Principles:

- A matrix is an array where each element is also an array
- First index = row, second index = column
- All inner arrays should have the same length (rectangular matrix)
- Think visually: grid structure, not just nested data

Examples:

```
// 3x3 matrix (3 rows, 3 columns)
let grid: number[][] = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];

// Accessing elements
console.log(grid[0][0]); // 1 (first row, first column)
console.log(grid[1][2]); // 6 (second row, third column)
console.log(grid[2][1]); // 8 (third row, second column)

// Real-world: seating chart
```

```
let seats: string[][] = [
  ["Alice", "Bob", "Charlie"],
  ["David", "Emma", "Frank"],
  ["Grace", "Henry", "Iris"],
];
```

03.1 Working with Matrices 💻 (~600 words)

Learning Objectives:

- Create matrices programmatically
- Add and remove rows from matrices
- Modify matrix elements
- Understand matrix dimensions and size

Content Outline:

- Creating matrices: literal notation
- Adding rows: `.push()` entire arrays
- Removing rows: `.pop()` and `.shift()`
- Modifying cells: `matrix[row][col] = value`
- Matrix dimensions: rows and columns count

Transitions: Now that you can create and modify matrices, let's learn how to iterate through them systematically.

Key Principles:

- Each row is an array pushed to the matrix
- Adding columns requires modifying each row
- Matrix size: `rows × columns` format
- Keep matrices rectangular (equal row lengths)

Examples:

```
// Create empty matrix and build it
let schedule: string[][] = [];
schedule.push(["Math", "English", "Science"]);
schedule.push(["History", "Art", "PE"]);
schedule.push(["Spanish", "Music", "Lunch"]);

// Add a new row (new day)
schedule.push(["Assembly", "Reading", "Drama"]);

// Modify a cell
schedule[1][2] = "Computer Science"; // Change Art to CS

// Remove last row
let removedDay = schedule.pop();
```

Hands-On Exercise: Task: Build a movie theater seating system

1. Create a 4x5 matrix (4 rows, 5 seats each) filled with "Empty"
2. Reserve specific seats by changing them to "Reserved"
    - Row 0, Seat 2
    - Row 1, Seat 1 and Seat 3
    - Row 3, Seat 4
3. Print the entire matrix to visualize the seating
4. Add a new row (5 new seats, all "Empty")
5. Print the updated matrix

Success Criteria:

- Initial matrix is 4x5 with all "Empty"
- Specified seats changed to "Reserved"
- New row added correctly
- Output shows clear matrix structure

03.2 Matrix Iteration and Access 💻 (~650 words)

Learning Objectives:

- Use nested loops to traverse matrices
- Process all elements in a 2D array
- Perform row-wise and column-wise operations
- Apply iteration patterns to real matrix problems

Content Outline:

- Nested loop pattern: outer for rows, inner for columns
- Row-wise processing: iterating through rows
- Column-wise processing: iterating through columns
- Common operations: sum all elements, find values, count occurrences
- Edge cases: empty matrices, single row/column

Transitions: With matrix iteration mastered, we move to sorting and searching—techniques for organizing and finding data efficiently.

Key Principles:

- Outer loop controls rows (`i`), inner loop controls columns (`j`)
- Pattern: `for (i = 0; i < matrix.length; i++)` for rows
- Pattern: `for (j = 0; j < matrix[i].length; j++)` for columns
- Always access as `matrix[i][j]`, never reverse

Examples:

```
let scores: number[][] = [
  [85, 92, 78],
  [90, 88, 95],
  [76, 84, 89],
];

// Print all elements
for (let i = 0; i < scores.length; i++) {
  for (let j = 0; j < scores[i].length; j++) {
    console.log(`Student ${i}, Test ${j}: ${scores[i][j]}`);
  }
}

// Calculate total of all scores
let total = 0;
for (let i = 0; i < scores.length; i++) {
  for (let j = 0; j < scores[i].length; j++) {
    total += scores[i][j];
  }
}
```

Hands-On Exercise: Task: Analyze a tic-tac-toe board

1. Create a 3x3 matrix representing a tic-tac-toe board with values: "X", "O", or "-" (empty)
2. Fill it with a game in progress (at least 5 moves)
3. Count how many "X" marks are on the board
4. Count how many "O" marks are on the board
5. Count how many empty spaces ("-") remain
6. Print the board in a readable format
7. Print the counts

Success Criteria:

- 3x3 matrix with valid game state
- Counts accurate for X, O, and empty
- Board displays clearly (bonus: format as actual grid)
- All cells processed

Section 04: Sorting and Searching

04.0 Sorting Arrays 🕮 (~550 words)

Learning Objectives:

- Understand what sorting means and why it's important
- Use the `.sort()` method for numbers and strings
- Provide comparison functions for custom sorting
- Recognize when sorted data enables better algorithms

Content Outline:

- Definition: arranging elements in order
- `.sort()` method: default lexicographic sorting
- Comparison functions: `(a, b) => a - b` for numbers
- Ascending vs. descending order
- Why sorting matters: enables binary search, improves readability

Transitions: Sorted arrays are powerful tools. Next, we'll learn to search through unsorted arrays using linear search.

Key Principles:

- `.sort()` modifies the original array (mutation)
- Without comparison function, `.sort()` treats all as strings
- Comparison function returns negative, zero, or positive
- Sorting once can speed up many searches

Examples:

```
let numbers: number[] = [40, 10, 30, 20, 50];

// Wrong: sorts as strings
numbers.sort(); // [10, 20, 30, 40, 50] - works but unreliable

// Correct: provide comparison for numbers
numbers.sort((a, b) => a - b); // ascending: [10, 20, 30, 40, 50]
numbers.sort((a, b) => b - a); // descending: [50, 40, 30, 20, 10]

// Strings sort alphabetically by default
let names: string[] = ["Charlie", "Alice", "Bob"];
names.sort(); // ["Alice", "Bob", "Charlie"]
```

04.1 Linear Search in Arrays 💻 (~650 words)

Learning Objectives:

- Implement linear search from scratch
- Use `.find()` method for finding elements
- Use `.includes()` for existence checks
- Understand when to use each method

Content Outline:

- Linear search algorithm: check each element sequentially
- Implementation with loop and conditional
- `.find()`: returns first matching element
- `.includes()`: returns boolean
- When to use each method

Transitions: Linear search works on any array. Next, we'll learn binary search—a faster algorithm for sorted arrays.

Key Principles:

- Linear search checks every element until found
- Works on sorted and unsorted arrays
- `.find()` returns element or `undefined`
- Simple but not always fastest

Examples:

```
let inventory: string[] = ["apples", "bananas", "oranges", "grapes"];

// Manual linear search
function linearSearch(arr: string[], target: string): number {
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === target) {
      return i; // found at index i
    }
  }
  return -1; // not found
}

// Using .find()
let fruit = inventory.find((item) => item === "bananas"); // "bananas"

// Using .includes()
let hasOranges = inventory.includes("oranges"); // true
```

Hands-On Exercise: Task: Build a student lookup system

1. Create an array of 8 student names
2. Implement a manual linear search function that returns the index
3. Search for 3 different names (2 exist, 1 doesn't)
4. Use `.find()` to get a student object from this array:

   ```
   let students = [
     { name: "Ana", grade: 85 },
     { name: "Bob", grade: 92 },
     { name: "Carlos", grade: 78 },
   ];
   ```

5. Find the student with grade > 90
6. Print all results

Success Criteria:

- Manual search returns correct indices or -1

- `.find()` returns correct student object
- Grade search finds Bob
- Code handles not found cases

04.2 Binary Search Algorithm 🕮 (~600 words)

Learning Objectives:

- Understand how binary search works
- Recognize why sorted data is required
- Implement binary search step-by-step
- Compare binary search efficiency to linear search

Content Outline:

- Prerequisites: sorted arrays only
- Algorithm: divide and conquer approach
- Steps: find middle, compare, eliminate half
- Loop until found or range exhausted
- When binary is better than linear

Transitions: Binary search is powerful but requires sorted data. Next, we'll examine common pitfalls when working with arrays.

Key Principles:

- Binary search only works on sorted arrays
- Each comparison eliminates half the remaining elements
- Much faster than linear for large arrays
- Requires integer indices (low, mid, high)

Examples:

```
function binarySearch(arr: number[], target: number): number {
  let low = 0;
  let high = arr.length - 1;

  while (low <= high) {
    let mid = Math.floor((low + high) / 2);

    if (arr[mid] === target) {
      return mid; // found
    } else if (arr[mid] < target) {
      low = mid + 1; // search right half
    } else {
      high = mid - 1; // search left half
    }
  }

  return -1; // not found
```

```
}

// Example usage
let sorted: number[] = [10, 20, 30, 40, 50, 60, 70];
console.log(binarySearch(sorted, 40)); // 3
console.log(binarySearch(sorted, 25)); // -1
```

04.3 Common Array Pitfalls and Anti-Patterns 🕮 (~550 words)

Learning Objectives:

- Identify common mistakes when working with arrays
- Understand the consequences of each anti-pattern
- Recognize correct alternatives to problematic approaches
- Develop defensive programming habits with arrays

Content Outline:

- Hardcoding array length instead of using `.length`
- Mutating arrays during iteration
- Accessing out-of-bounds indices without checking
- Mixed-type arrays (violating homogeneity)
- Off-by-one errors in loops
- Infinite loops in array processing

Transitions: Avoiding these pitfalls will make your array code more robust. Finally, let's apply searching to matrices.

Key Principles:

- Always use `.length`, never hardcode limits
- Don't modify array size while iterating through it
- Check bounds before accessing: `if (i < arr.length)`
- Keep arrays homogeneous (same type)

Examples:

Anti-Pattern 1: Hardcoded Length

```
// ❌ Wrong
let numbers = [10, 20, 30];
for (let i = 0; i < 3; i++) {
  // breaks if array changes
  console.log(numbers[i]);
}

// ☑ Correct
for (let i = 0; i < numbers.length; i++) {
```

```
  console.log(numbers[i]);
}
```

Anti-Pattern 2: Mutation During Iteration

```
// ❌ Wrong - skips elements
let items = [1, 2, 3, 4, 5];
for (let i = 0; i < items.length; i++) {
  items.splice(i, 1); // removes during loop
}

// ☑ Correct - iterate backwards
for (let i = items.length - 1; i >= 0; i--) {
  items.splice(i, 1);
}
```

Anti-Pattern 3: No Bounds Checking

```
// ❌ Wrong
let scores = [85, 90, 78];
console.log(scores[100]); // undefined - no error!

// ☑ Correct
if (index >= 0 && index < scores.length) {
  console.log(scores[index]);
}
```

Anti-Pattern 4: Mixed Types

```
// ❌ Wrong
let mixedBag = [1, "hello", true, 3.14]; // confusing

// ☑ Correct - use separate arrays
let numbers: number[] = [1, 3.14];
let messages: string[] = ["hello"];
let flags: boolean[] = [true];
```

Consequences:

- Hardcoded lengths cause errors when arrays grow
- Mutation during iteration causes skipped/duplicate processing
- Out-of-bounds access returns `undefined` silently
- Mixed types make iteration and type-checking difficult

04.4 Searching in Matrices 💻 (~650 words)

Learning Objectives:

- Search for values in 2D arrays
- Find specific positions (row, column) of elements
- Implement row-wise and column-wise searches
- Handle "not found" cases in matrices

Content Outline:

- Linear search in matrices: nested loop approach
- Finding first occurrence: return position and exit
- Finding all occurrences: collecting positions
- Searching specific rows or columns
- Return formats: object with row/col, tuple, or -1

Transitions: You've now mastered arrays and matrices. Let's put all these skills to the test with practical challenges.

Key Principles:

- Matrix search requires nested loops
- Return position as `{row, col}` or `[row, col]`
- Stop early when first match found (efficiency)
- Return `null` or `-1` if not found

Examples:

```
function findInMatrix(
  matrix: number[][],
  target: number,
): { row: number; col: number } | null {
  for (let i = 0; i < matrix.length; i++) {
    for (let j = 0; j < matrix[i].length; j++) {
      if (matrix[i][j] === target) {
        return { row: i, col: j };
      }
    }
  }
  return null; // not found
}

// Example usage
let grid = [
  [10, 20, 30],
  [40, 50, 60],
  [70, 80, 90],
];

let position = findInMatrix(grid, 50);
// {row: 1, col: 1}
```

Hands-On Exercise: Task: Treasure hunt in a grid

1. Create a 5x5 matrix with random numbers (1-100)
2. Implement `findTreasure()` that searches for a specific number
3. Return the position as `{row, col}` or `null`
4. Search for 3 different values
5. Implement `findAllTreasures()` that finds ALL positions of a value
6. Place the same number in multiple cells and find all locations
7. Print results for each search

Success Criteria:

- Search function finds correct positions
- Returns `null` when value not found
- `findAllTreasures()` returns array of all positions
- Code stops early when finding first occurrence (for single search)
- Clear output showing found positions

Section 05: Practice and Assessment

05.0 Array Manipulation Challenges 💻 (~650 words)

Learning Objectives:

- Combine multiple array operations to solve complex problems
- Choose appropriate methods for each subtask
- Debug array-related issues systematically
- Build confidence with practical array challenges

Content Outline:

- Challenge 1: Array transformation pipeline
- Challenge 2: Data filtering and aggregation
- Challenge 3: Array restructuring
- Debugging strategies for array code
- Performance considerations

Transitions: These challenges integrate everything you've learned. Next, we'll examine how to approach real-world array problems systematically.

Key Principles:

- Break complex problems into smaller array operations
- Chain methods when appropriate: `.filter().map()`
- Test with simple data first, then scale up
- Use descriptive variable names for clarity

Hands-On Exercise:

Challenge 1: Student Grade Processor Given an array of student objects:

```
let students = [
  { name: "Ana", scores: [85, 90, 78] },
  { name: "Bob", scores: [92, 88, 95] },
  { name: "Carlos", scores: [76, 84, 89] },
];
```

1. Calculate each student's average score
2. Create a new array with student names and averages
3. Sort students by average (highest first)
4. Filter students with average ≥ 85
5. Print the honor roll

Challenge 2: Inventory Management Given product inventory:

```
let inventory = [
  { product: "Laptop", quantity: 5, price: 999 },
  { product: "Mouse", quantity: 25, price: 25 },
  { product: "Keyboard", quantity: 15, price: 75 },
];
```

1. Find total inventory value (sum of quantity × price)
2. Find most expensive product
3. Filter products with quantity < 10
4. Sort by quantity (lowest first)
5. Create a restock list (quantity < 10)

Challenge 3: Matrix Operations

1. Create a 4x4 matrix with numbers 1-16
2. Calculate sum of each row
3. Calculate sum of each column
4. Find the maximum value in the entire matrix
5. Count how many numbers are even

Success Criteria:

- All calculations accurate
- Appropriate array methods used
- Code is readable and well-structured
- Edge cases handled (empty arrays, zeros)

05.1 Solving Real-World Array Problems 🕮 (~700 words)

Learning Objectives:

- Understand systematic approaches to array-based problems

- Recognize common problem patterns and their solutions
- Learn to break down complex scenarios into array operations
- Apply appropriate algorithms for different problem types

Content Outline:

- Problem-solving methodology for array challenges
- Scenario 1 analysis: E-commerce cart system
- Scenario 2 analysis: Schedule conflict detection
- Scenario 3 analysis: Data deduplication
- Scenario 4 analysis: Leaderboard management

Transitions: Understanding these solution patterns prepares you for any array challenge. Now let's assess your complete understanding.

Key Principles:

- Identify the core data structure (what does each array hold?)
- Determine required operations (search, filter, transform, aggregate)
- Choose the right iteration method for each step
- Consider edge cases before writing code

Scenario 1: Shopping Cart System

Problem: Build a cart that calculates totals, applies discounts, and manages items.

Solution Approach:

1. **Data structure**: Array of objects `{name, price, quantity}`
2. **Item total**: Use `.map()` to create array of `price * quantity`
3. **Cart total**: Use `.reduce()` to sum all item totals
4. **Apply discount**: Use `.map()` with conditional (if price > 50, apply 10% off)
5. **Remove empty items**: Use `.filter()` to keep only `quantity > 0`
6. **Sort by value**: Use `.sort()` with comparison function on item totals

Key operations:

- `.map()` for transformations (calculating item totals, applying discounts)
- `.reduce()` for aggregation (sum all values)
- `.filter()` for selection (remove empties)
- `.sort()` for ordering

Scenario 2: Meeting Scheduler

Problem: Detect conflicts in meeting schedules and find free time slots.

Data structure:

```
let meetings = [
  { title: "Standup", start: 9, end: 10 },
```

```
  { title: "Design Review", start: 11, end: 12 },
];
```

Solution Approach:

1. **Check conflict**: Loop through existing meetings, compare new meeting's start/end times
   - Conflict exists if: `newStart < existingEnd && newEnd > existingStart`
2. **Find free slots**:
   - Sort meetings by start time first
   - Iterate through sorted meetings
   - Gap exists between `meetings[i].end` and `meetings[i+1].start`
3. **Sort meetings**: Use `.sort((a, b) => a.start - b.start)`
4. **Total hours**: Use `.reduce()` to sum `(end - start)` for all meetings
5. **Longest meeting**: Use `.reduce()` or loop to track max `(end - start)`

Key operations:

- Manual loop with conditionals for conflict detection
- `.sort()` for chronological order
- `.reduce()` for totals
- Comparison logic for finding maximums

Scenario 3: Duplicate Remover

Problem: Clean arrays by removing duplicate values.

Solution Approach for Numbers:

1. **Simple deduplication**: Use `Set` then convert back to array

```
let unique = [...new Set(numbers)];
```

2. **Count occurrences**: Use object/map to track counts

```
let counts = {};
numbers.forEach((num) => (counts[num] = (counts[num] || 0) + 1));
```

3. **Keep first occurrence**: Use `.filter()` with `.indexOf()`

```
let unique = arr.filter((item, index) => arr.indexOf(item) === index);
```

Solution Approach for Objects:

1. **Deduplicate by ID**: Use `.filter()` checking if current index matches first index of that ID

2. **Use Map**: Track seen IDs in a Set, filter based on Set membership
3. **Keep only duplicates**: Filter items where count > 1

Key operations:

- Set for automatic deduplication
- `.filter()` with `.indexOf()` for manual control
- Object/Map for counting occurrences

Scenario 4: Leaderboard System

Problem: Maintain top scores, insert new scores correctly, handle ties.

Solution Approach:

1. **Track top 10**: Keep array sorted, slice to keep only first 10
2. **Insert new score**:
    - Push new score to array
    - Sort descending: `.sort((a, b) => b - a)`
    - Keep top 10: `leaderboard = leaderboard.slice(0, 10)`
3. **Handle ties**: Sort by score first, then by timestamp if equal
4. **Calculate rank**:
    - Sort all scores descending
    - Use `.findIndex()` to locate the score
    - Rank = index + 1
5. **Find median**:
    - Sort array
    - If odd length: middle element
    - If even length: average of two middle elements

Key operations:

- `.sort()` with comparison for ordering
- `.slice()` for limiting size
- `.findIndex()` for finding position
- Manual calculation for median (requires understanding middle position)

Common Problem-Solving Pattern:

1. **Understand the data**: What does each array element represent?
2. **Identify the goal**: Search? Transform? Aggregate? Filter?
3. **Choose methods**:
    - Search → `.find()`, `.includes()`, `.indexOf()`
    - Transform → `.map()`
    - Filter → `.filter()`
    - Aggregate → `.reduce()` or manual loop
    - Sort → `.sort()`
4. **Handle edge cases**: Empty arrays, single elements, all same values

5. **Test incrementally**: Start with simple data, verify each step

---

05.2 Arrays and Collections Assessment 😤 (~700 words)

Learning Objectives:

- Demonstrate comprehension of array fundamentals
- Apply array methods to solve problems
- Analyze code for correctness and efficiency
- Evaluate different approaches to array operations

Question Format: Multiple choice (7 questions)

Evaluation Criteria:

- 6-7 correct: Excellent - Ready to advance
- 4-5 correct: Good - Review challenging topics
- 2-3 correct: Needs review - Revisit lessons
- 0-1 correct: Requires additional practice

Question 1: Array Fundamentals

What will the following code output?

```
let numbers: number[] = [10, 20, 30];
numbers.push(40);
numbers.unshift(5);
console.log(numbers[2]);
```

A) 10 B) 20 C) 30 D) 40

Correct Answer: B) 20 **Explanation:** After push(40), array is [10, 20, 30, 40]. After unshift(5), array is [5, 10, 20, 30, 40]. Index 2 is 20.

Question 2: Array Methods

Which method should you use to check if an array contains a specific value and get a true/false result?

A) `.indexOf()` B) `.includes()` C) `.find()` D) `.filter()`

Correct Answer: B) `.includes()` **Explanation:** `.includes()` returns boolean. `.indexOf()` returns index. `.find()` returns element. `.filter()` returns array.

Question 3: Iteration

What is the output of this code?

```
let values = [2, 4, 6, 8];
let result = values.map((x) => x * 2);
console.log(values.length === result.length);
```

A) true B) false C) undefined D) Error

Correct Answer: A) true **Explanation:** `.map()` creates new array with same length as original, transforming each element.

Question 4: Matrix Access

Given this matrix:

```
let grid = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];
```

What is `grid[2][0]`?

A) 3 B) 6 C) 7 D) 9

Correct Answer: C) 7 **Explanation:** grid[2] is the third row [7, 8, 9], and [0] is first element of that row.

Question 5: Sorting

What's wrong with this code?

```
let numbers = [100, 20, 3];
numbers.sort();
console.log(numbers);
```

A) Nothing, it works correctly B) sort() without comparison function treats numbers as strings C) sort() doesn't modify the original array D) sort() requires two parameters

Correct Answer: B) sort() without comparison function treats numbers as strings **Explanation:** Without comparison function, [100, 20, 3] sorts as ["100", "20", "3"] → [100, 20, 3] (wrong order).

Question 6: Searching

Binary search requires:

A) Any array B) A sorted array C) An array with unique values D) An array with less than 100 elements

Correct Answer: B) A sorted array **Explanation:** Binary search divides sorted arrays in half. Doesn't work on unsorted data.

Question 7: Anti-Patterns

Which is the correct way to iterate through an array?

A) `for (let i = 0; i <= arr.length; i++)` B) `for (let i = 0; i < arr.length; i++)` C) `for (let i = 1; i <= arr.length; i++)` D) `while (true) { arr[i]; i++; }`

Correct Answer: B) `for (let i = 0; i < arr.length; i++)` **Explanation:** Starts at 0 (first index), stops before length (prevents out-of-bounds). Option A would access arr[length] which is undefined.

Score Interpretation:

6-7 correct: Excellent understanding! You're ready for objects and more advanced data structures. Key strengths: array fundamentals, iteration, and searching algorithms.

4-5 correct: Good grasp of arrays. Review: matrix indexing, array method differences (.map vs .filter vs .find), and binary search requirements.

2-3 correct: Foundational understanding present. Needs review: array indexing (0-based), method return values, sorting behavior, and iteration patterns. Practice hands-on exercises.

0-1 correct: Requires additional practice. Revisit: array creation and access, basic methods (push, pop, indexOf), simple iteration with for loops. Work through Section 1-2 exercises again.

Section 06: Conclusion

06.0 Mastering Collections: Next Steps (~425 words)

Content Outline:

What You've Accomplished: You've mastered one of programming's fundamental building blocks—arrays. You started with simple collections, learned to traverse them efficiently, expanded into multi-dimensional matrices, and discovered powerful algorithms for sorting and searching. These skills form the foundation for every data-driven application you'll build.

Key Skills Acquired:

- Creating and manipulating arrays with confidence
- Choosing the right iteration method for each task
- Working with complex multi-dimensional data structures
- Implementing searching and sorting algorithms
- Avoiding common array pitfalls through defensive coding

Connection to the Bigger Picture: Arrays store multiple values of the same type. But real-world applications need to represent complex entities: a user has a name, email, and preferences; a product has a title, price, and

inventory. This is where objects come in—the next major concept in your journey. You'll soon combine arrays and objects to build sophisticated data models: arrays of user objects, product catalogs, and more.

The iteration patterns you learned here (forEach, map, filter) will work on objects too. The searching techniques will help you find specific objects in collections. Everything builds on everything.

Next Steps:

Immediate Practice:

- Solve 3-5 more array challenges from coding platforms (LeetCode Easy, Codewars)
- Refactor old code to use modern array methods instead of traditional loops
- Build a mini-project: todo list manager, grade calculator, or inventory system

Prepare for Objects:

- Review how you've been grouping related variables (firstName, lastName, email)
- Think about entities in your favorite apps (users, posts, products)
- Recognize that objects will organize data by meaning, not just position

Deepen Your Knowledge:

- Explore additional array methods: `.reduce()`, `.some()`, `.every()`
- Learn about advanced sorting (custom comparisons for objects)
- Investigate algorithm visualization tools

Resources for Continued Learning:

- JavaScript.info - Arrays chapter (comprehensive reference)
- Visualgo.net - Algorithm visualizations (sorting, searching)
- Array method practice on FreeCodeCamp
- TypeScript handbook - Working with arrays

Final Encouragement: Arrays might seem simple, but you've learned powerful techniques. Professional developers use these exact patterns every day. The difference between a beginner and an expert isn't knowing obscure methods—it's choosing the right approach, writing clean code, and avoiding pitfalls. You're well on your way.

Keep practicing. Keep building. Next up: objects will unlock a whole new level of data organization. You're ready.